ONE Namah Shivay

Customer churn Predication

① Problem Understanding

- Why churn Predication
  - customer acquire karna expensive hota hai
  - Existing customer ko retain karna cheap hota hai
  - Business wants :- Kaun churn karega pehle se btao.

- I built a full stack customer churn prediction system using machine learning. It predicts whether a telecom customer is likely to churn, and i integrated the model with a FASTAPI backend and a React frontend for real-time predictions.

② Dataset :- Telco Customer Churn

- I used the Telco customer churn dataset, which contains around 7000 customer records with 21 features describing customer demographics, services & account info

- columns Explaration
  - ① customer ID → Unique ID for each cust. → ML me use nhi kiya/drop kiya
  - ② Gender :- demographic feature
  - ③ Senior Citizen :- 1-Senior, 0-not → Age Based behaviour diff
  - ④ Partner :- yes/No OR Married ya not
  - ⑤ Dependents :- Yes/No → family responsibility
  - ⑥ Tenure :- No. of months customer stayed
    - ★ Long tenure = Loyal customer
  - ① Phone service :- yes/No → Yes phone connection or not
  - ⑧ Internet service :- DSL/Fiber optic/No
    - 1-Fiber optic → high churn
  - ⑨ online Security → Yes/No → security sen
  - ⑩ Online backup → Yes/No → backup facility
  - ⑪ Contract :- month-to-month/one year/two year
    - ★ month-to-month → high churn
  - ⑦ Device Protection :- Yes/No → Protection service
  - ⑫ Tech support :- Yes/No → No tech support / high churn
  - ⑬ Monthly charges → higher charges → high churn
  - ⑭ Total charges :- Total amount paid → tenure × monthly charges approx
  - ⑰ churn
    - Yes :- customer left
    - No :- customer stayed

- Some feature like contract type, tenure and monthly charges have a strong impact on churn, while demographic feature like gender have relatively less influence.

- Feature Types → ① Numerical
  - tenure
  - monthly charges
  - total charges

  ② Categorical
  - contract
  - internet service
  - Payment method

→ Large column transformer use kiye
is a tool in scikit learn that allows us to apply different preprocessing steps to different columns of a dataset

→ Simple meaning :-
Alag-Alag column pe alag treatment
Numeric → scaling
Categorical → Encoding
- OneHot encoder
- Standard Scaler

→ Pipeline :- is a sequence of steps where data is processed and passed through preprocessing and model training in a fixed order.

Raw Data → Preprocessing → Model → Prediction

- Without Pipeline :- Training aur testing me different preprocessing
  - Data leakage risk
  - Messy code

→ ColumnTransformer → feature preprocessing | Pipeline → full workflow

⇒ Data Leakage :- happens when information from the test data is accidentally used during training, leading to overly optimistic and incorrect model performance.

→ Types:
- ① Preprocessing leakage
  - → Scaling before split
  - → Encoding before split
- ② Target Leakage :- Fake feature future info data lial
  - Ex :- Account closed Date :- churn predict karne
  - → obviously model cheat karega kya sir
- ③ Time Leakage :- Future data use ho rha past predict karne me
  - Ex) Tomorrow's data use to predict today

→ Why Pipeline solves this :- Pipeline + ColumnTransformer
- → Yes ensure karta hai
  - → fit :- only on training data
  - → transform :- on test data
- → leakage impossible ho jata hai

Real Impact
without fixing leakage
- Accuracy :- high (fake)
- Real Perfo :- low (bad)

→ I avoided data leakage by using a pipeline, ensuring that preprocessing steps like scaling and encoding were applied only on the training data and not on the test data.

→ Short version :- Data Leakage = Model cheating

⇒ ROC-AUC
↳ Measures how well a model ka distinguish between two classes, jaise, churn and non-churn customers.
↳ Simple Meaning → Model kitna acha hai Yes aur No ko alag karne me

• ROC :- Receiver operating characteristic
AUC :- Area under curve

• If AUC is 0.94 then Excellent

• ROC-AUC :- Probability that model ranks a churn customer higher than a non-churn customer

→ I used ROC-AUC as the evaluation metric because the dataset was imbalanced, and it measures the model's ability to distinguish b/w churn & non-churn customer across all thresholds.

⇒ (1) Precision :- $\frac{TP}{TP+FP}$ | Ex:- Model bole 100 churn → Actually churn = 70 → Precision = 70%
↳ jitne logon ko churn bola, unme se kitne actually churn the

⇒ (2) Recall :- $\frac{TP}{TP+FN}$ | Ex:- 100 actual churn, Model ne 80 detect kiye, Recall = 80%
↳ Actual churn customer me se kitne pakde

⇒ (3) F1-score :- F1 = 2 × (Precision × Recall) / (Precision + Recall)
↳ Balance between precision & recall

⇒ TP, FP, TN, FN :- TP → True positive
FP → False positive
TN → True negative
FN → False negative
↳ Confusion matrix Basics :-

	Predicted Yes	Predicted No
Actual Yes	TP	FN
Actual No	FP	TN

(3) Data Preprocessing

↳ raw data is not directly suitable for machine learning models, so, preprocessing is required to clean, transform and prepare the data for training.

↳ (1) Drop customer ID
(2) convert TotalCharges to numeric
(3) Handle missing value → drop us kiya
↳ Alternate - fill with mean, mode
(4) Encode target variable
↳ because model no-string + target
(5) separate features & target → X → input
Y → output
(6) train-test split
↳ test data unseen hona chahiye
stratify → class balance maintain
(7) column Transformer
(8) pipeline

(4) Model selection

↳ The goal was to select a model that can accurately predict customer churn while maintaining good generalization and interpretability.

→ We ~~use~~ don't use only one single model
↳ multiple models try kiye

↳ models tried → Logistic Regression → final
→ Random forest
→ XGBoost

→ I experimental with multiple models to compare their performance and select the best one based on evaluation metrices like ROC-AUC

Logistic Reg
very ✓
Reason 1 → Data Nature :- dataset mostly linear relationship
↳ linear model :- tenure ↑ → churn ↓
perfectly fit kiya
charges ↑ → churn ↑

↳ Reason 2 → Categorical features :- many one-hot encoded features
↳ logistic handles them well

↳ Reason 3 → Simplicity → fast
→ easy to deploy
↳ less overfitting

↳ Reason 4 → Interpretability :- coefficient switch
↳ feature importance easily

Why not Random forest → slightly lower AUC
↳ overfitting risk
↳ hard to interpret

Why not XGBoost → Powerful model
↳ But improvement minimal
↳ complexity high

Step ⑤ :- Model Evaluation

- The goal of model evaluation is to measure how well the trained model performs on unseen data.
- Why we use ROC-AUC → I used ROC-AUC as the primary evaluation metric because it is most suitable for imbalanced & measure the model's ability to distinguish b/w classes.
- Cross Validation :- 5k split unseen data hai
  - CV mei → Data 5 parts me split → Har part test banta hai
  - Benefits :-
    - More reliable score
    - Overfitting detect hota hai
    - Stability check hoti hai
- Interpretation :- Test AUC = 0.836, CV AUC = 0.845
  - Close values = good model
  - Agar gap bada hota → overfitting

Confusion Matrix use kiya
- TP, FP, FN, TN
- Precision
- Recall

Step ⑥ :- Threshold Tuning

→ 6.1 :- Default Behavior
- Model output deta hai
  - Prob. b/w 0 to 1
  - Ex:- 0.88 → churn, 0.23 → no churn
- Default Rule :- threshold = 0.5
  - Agar ≥ 0.5 → churn, < 0.5 → no churn

(lower threshold) more cust. classified as churn

Ye always best nahi hota

Step ⑦ :- Feature Importance

:- The goal of feature importance is to understand which features have the most influence on the model's predictions.

- Why → ML model sirf prediction nahi karte → Humeye bhi samjhana hai
- We use → logistic regression coefficients = feature importance
  - Positive coefficient → churn increase
  - Negative coefficient → churn decrease

Step ⑧ :- Model Saving

:- goal is to store the trained model so it can be reused for predictions without retraining.

Step ⑨ :- FastAPI Backend

:- convert the trained ~~model~~ ML into an API so that it can be accessed by external apps like a frontend or other service

- Why Backend needed → ML model ek file mei padha hai (.pkl)
  - usko use karne ke liye :- frontend → model connect karna padta hai

Step ⑩ :- React Frontend

goal was to build a user friendly interface where users can input customer details and get real time churn prediction from the model

• Why Frontend Needed
- Backend API bana gyi
- But user directly API use nhi karunga
- User wants :- Simple UI + Input + result

→ Why react?
- Because
  - interactive UI
  - fast updates
  - Component based
  - Industry standard

→ UI me
- Tenure input
- Monthly charge input
- Predict button
- Result display

→ Core logic
- Step 1 :- User Input
- Step 2 :- API call
- Step 3 :- Response receive

→ Full flow :- User enters data → React sends request (Axios) → FastAPI recieve → Model predicts → Response back → React displays result